Assignment 2: Data Analytics and Machine Learning

Group ____

Member 1 (Student ID) · Member 2 (Student ID) · Member 3 (Student ID)

8 October 2026

The complete R code is in the accompanying script `Assignment2.R`*. Code is shown below for Questions 2 and 3; for Question 6 only the results are shown, to keep the report readable.*

Question 1: Bias–variance decomposition

Fix a point x. The true value $f(x)$ is a constant, while the estimate $\hat{f}(x)$ is random because it depends on the training sample that was drawn. Write $\bar{f}(x) = E[\hat{f}(x)]$, which is also a constant. Adding and subtracting $\bar{f}(x)$:

$$\begin{aligned} E[(f(x) - \hat{f}(x))^2] &= E\Big[\big(\underbrace{f(x) - \bar{f}(x)}_{\text{constant}} + \bar{f}(x) - \hat{f}(x)\big)^2\Big] \\ &= \big(f(x) - \bar{f}(x)\big)^2 + 2\big(f(x) - \bar{f}(x)\big)\,\underbrace{E[\bar{f}(x) - \hat{f}(x)]}_{=\,\bar{f}(x) - \bar{f}(x) \,=\, 0} + E\Big[\big(\hat{f}(x) - \bar{f}(x)\big)^2\Big] \\ &= \big[f(x) - E\hat{f}(x)\big]^2 + \mathrm{Var}\left[\hat{f}(x)\right] \;=\; \mathrm{Bias}^2[\hat{f}(x)] + \mathrm{Var}\left[\hat{f}(x)\right]. \end{aligned}$$

The cross term vanishes because $f(x) - \bar{f}(x)$ is a constant and $\hat{f}(x)$ deviates from its own mean by zero on average. If we compare the prediction with a noisy observation $y = f(x) + \varepsilon$, where $E[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$ and ε is independent of the training data, the same argument gives $E[(y - \hat{f}(x))^2] = \mathrm{Bias}^2 + \mathrm{Variance} + \sigma^2$. The term σ^2 is the irreducible error that no model can remove.

How we use the decomposition in machine learning

- **It explains the bias–variance trade-off.** A more flexible model (a deeper tree, a smaller k in KNN, QDA instead of LDA, more polynomial terms) reduces bias but increases variance, because it starts fitting noise in the particular training sample. Test error is therefore U-shaped in model complexity, while training error keeps falling. This is why a model must never be chosen on training error.
- **It guides model selection and tuning.** We choose the complexity that minimises *estimated test error*, using a validation set or cross-validation. Examples are k in KNN, the number of features tried at each split of a random forest, and the complexity parameter `cp` of a tree. We do exactly this in Question 3(e) and Question 6.
- **It helps diagnose a model and choose the fix.**
  - High training error and high test error mean high bias (underfitting). We need more informative features or a more flexible model.
  - Low training error but much higher test error means high variance (overfitting). We need more data, regularisation, pruning, fewer features, or averaging. Bagging and random

forests work mainly by reducing variance; boosting mainly reduces bias.

Question 2: Confusion matrix for a credit-risk model

We treat **Default as the positive class**. The table gives TP = 700, FN = 400 (defaulters predicted as non-default), FP = 100 and TN = 56,000.

```
TN <- 56000   # predicted non-default, truly non-default
FN <- 400     # predicted non-default, truly default   (missed defaulters)
FP <- 100     # predicted default,     truly non-default (false alarms)
TP <- 700     # predicted default,     truly default

conf_mat <- matrix(c(TN, FP, FN, TP), nrow = 2,
                   dimnames = list(Predicted = c("Non-default (-)", "Default (+)"),
                                   True      = c("Non-default (-)", "Default (+)")))
conf_mat
```

```
#>                  True
#> Predicted         Non-default (-) Default (+)
#>   Non-default (-)           56000         400
#>   Default (+)                 100         700
```

```
N <- TN + FN + FP + TP
q2_metrics <- c(
  "Accuracy"                          = (TP + TN) / N,
  "Sensitivity (recall, TPR)"         = TP / (TP + FN),
  "Specificity (TNR)"                 = TN / (TN + FP),
  "Precision (PPV)"                   = TP / (TP + FP),
  "Miss rate (FNR = 1 - sens.)"       = FN / (TP + FN),
  "Default rate in data (prevalence)" = (TP + FN) / N,
  "Baseline accuracy: predict all non-default" = (TN + FP) / N,
  "Balanced accuracy"                 = (TP / (TP + FN) + TN / (TN + FP)) / 2
)
q2_table <- data.frame(Metric = names(q2_metrics), Value = pct(q2_metrics, 2), row.names = NULL)
q2_table
```

```
#>                                    Metric  Value
#> 1                                Accuracy 99.13%
#> 2                  Sensitivity (recall, TPR) 63.64%
#> 3                          Specificity (TNR) 99.82%
#> 4                            Precision (PPV) 87.50%
#> 5                Miss rate (FNR = 1 - sens.) 36.36%
#> 6          Default rate in data (prevalence)  1.92%
#> 7 Baseline accuracy: predict all non-default 98.08%
#> 8                          Balanced accuracy 81.73%
```

Sensitivity $= TP/(TP + FN) = 700/1{,}100 =$ **63.64%**, and **accuracy** $= (TP + TN)/N = 56{,}700/57{,}200 =$ **99.13%**.

Is this a good model? For a risk assessor, no, or at least not good enough.

- **The high accuracy is misleading because the classes are very imbalanced.** Only 1.92% of borrowers default. A useless rule that labels *everyone* "non-default" would already reach 98.08% accuracy. The model improves on that by only about one percentage point.

- **The model misses more than a third of the defaulters.** Sensitivity is only 63.6%, so 400 of the 1,100 actual defaulters (36.4%) would be classified as safe and would receive credit. In credit risk a false negative (lending to a defaulter) usually costs far more than a false positive (turning away a good customer): the bank loses principal rather than just some interest income.
- **The model is not without merit.** Specificity is 99.82% and precision is 87.5%, so when it flags a borrower it is usually right. The problem is that its threshold is too conservative.
- **How to improve it.** Lower the classification threshold, or use cost-sensitive weights or resampling of the minority class, to raise sensitivity at the price of some specificity. Then judge the model with metrics that reflect the costs: an ROC curve, precision–recall, or the expected loss. Raw accuracy is the wrong metric here.

Question 3: Graduate admission data

```
adm <- as.data.frame(read_excel(find_data("admission.xls")))
adm$rank  <- factor(adm$rank, levels = 1:4)            # 1 = most prestigious
adm$admit <- factor(adm$admit, levels = c(0, 1), labels = c("Rejected", "Admitted"))
str(adm)
summary(adm)
```

(a) Relationships among GRE, GPA and rank

```
# (a) Relationships among GRE, GPA and rank
rank_cols <- c("1" = "#0072B2", "2" = "#009E73", "3" = "#E69F00", "4" = "#D55E00")

p_scatter <- ggplot(adm, aes(gpa, gre, colour = rank)) +
  geom_point(position = position_jitter(width = 0.01, height = 4, seed = 1),
             alpha = 0.55, size = 1.5) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, linewidth = 0.9) +
  scale_colour_manual(values = rank_cols, name = "Institution rank") +
  labs(title = "GRE vs GPA (lines: linear fit within each rank)", x = "GPA", y = "GRE") +
  theme(legend.position = "right")

p_gpa <- ggplot(adm, aes(rank, gpa, fill = rank)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.8) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.2, fill = "white") +
  scale_fill_manual(values = rank_cols, guide = "none") +
  labs(title = "GPA by rank (diamond = mean)", x = "Institution rank", y = "GPA")

p_gre <- ggplot(adm, aes(rank, gre, fill = rank)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.8) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2.2, fill = "white") +
  scale_fill_manual(values = rank_cols, guide = "none") +
  labs(title = "GRE by rank (diamond = mean)", x = "Institution rank", y = "GRE")

fig_q3a <- p_scatter / (p_gpa | p_gre) + plot_layout(heights = c(1.15, 1))
fig_q3a
```

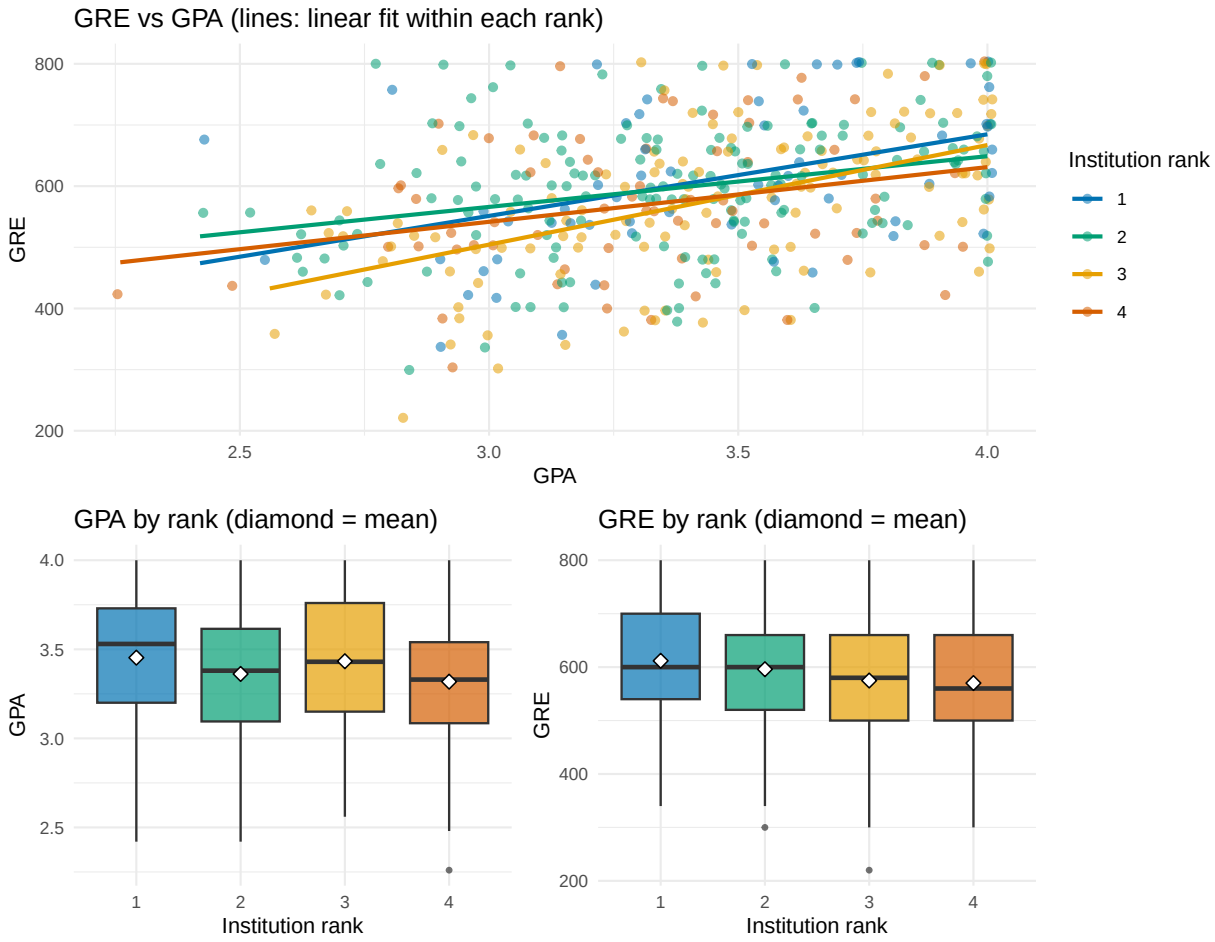


Figure 1: Relationships among GRE, GPA and institution rank (400 applicants).

```
save_plot(fig_q3a, "q3a_features.png", width = 8, height = 6.5)

# Correlations that support the picture
q3a_cor <- c(
  "Pearson r(GRE, GPA)"    = cor(adm$gre, adm$gpa),
  "Spearman rho(rank, GPA)" = cor(as.numeric(adm$rank), adm$gpa, method = "spearman"),
  "Spearman rho(rank, GRE)" = cor(as.numeric(adm$rank), adm$gre, method = "spearman")
)
round(q3a_cor, 3)
```

```
#>     Pearson r(GRE, GPA) Spearman rho(rank, GPA) Spearman rho(rank, GRE)
#>                   0.384                  -0.049                  -0.121
```

```
cor.test(adm$gre, adm$gpa)$p.value
```

```
#> [1] 1.595774e-15
```

Findings.

- **GRE and GPA are moderately positively correlated** ($r = 0.38$, $p < 0.001$). Applicants with a higher GPA also tend to score higher on the GRE, but the scatter is wide, so the two scores carry partly different information.
- **Rank is only weakly related to the two scores.** Mean GRE falls steadily from rank 1 (612) to rank 4 (570). GPA shows no monotone pattern: rank 3 has a higher mean GPA than

rank 2. The box plots overlap heavily (Spearman correlation of rank with GPA is -0.05 and with GRE is -0.12).

- **The GRE–GPA slope is similar in every rank**, so there is no strong interaction between rank and the scores.
- **Overall**, prestige of the undergraduate institution is not simply a proxy for academic ability. The three features can contribute separately to an admission decision.

(b) Average GPA by institution rank

```
# (b) Average GPA by institution rank
gpa_by_rank <- adm |>
  group_by(rank) |>
  summarise(applicants = n(), mean_GPA = mean(gpa), mean_GRE = mean(gre), .groups = "drop")
gpa_by_rank
```

Table 1: Average GPA (and GRE) by institution rank

Rank	Applicants	Mean GPA	Mean GRE
1	61	3.453	611.8
2	151	3.362	596.0
3	121	3.433	574.9
4	67	3.318	570.1

Mean GPA is between 3.32 and 3.45 in every group. Rank 1 is highest (3.453) and rank 4 is lowest (3.318), but the differences are small compared with the spread within each group.

(c) Percentage of applicants accepted from each rank

```
# (c) Percentage of applicants accepted from each rank
admit_by_rank <- adm |>
  group_by(rank) |>
  summarise(applicants = n(),
            admitted   = sum(admit == "Admitted"),
            admit_rate = 100 * mean(admit == "Admitted"),
            .groups = "drop")
admit_by_rank
overall_admit_rate <- mean(adm$admit == "Admitted")
```

Table 2: Admission rate by institution rank

Rank	Applicants	Admitted	Admitted (%)
1	61	33	54.1
2	151	54	35.8
3	121	28	23.1
4	67	12	17.9

The admission rate falls steadily with prestige, from 54.1% for rank 1 to 17.9% for rank 4 (overall 31.75%). Part (b) showed that GPA hardly differs across ranks. The large gap in admission rates therefore suggests that the committee values the prestige of the undergraduate institution in itself.

(d) Decision tree

```
# (d) Classification tree with GPA, rank and GRE (default settings: cp = 0.01)
set.seed(2026)   # rpart's 10-fold cross-validation (xerror) uses random folds
tree_adm <- rpart(admit ~ gpa + rank + gre, data = adm, method = "class")
print(tree_adm)
```

```
#> n= 400
#>
#> node), split, n, loss, yval, (yprob)
#>       * denotes terminal node
#>
#>   1) root 400 127 Rejected (0.6825000 0.3175000)
#>     2) gpa< 3.415 208  45 Rejected (0.7836538 0.2163462)
#>       4) rank=3,4 99  13 Rejected (0.8686869 0.1313131) *
#>       5) rank=1,2 109  32 Rejected (0.7064220 0.2935780)
#>        10) gre< 730 99  25 Rejected (0.7474747 0.2525253) *
#>        11) gre>=730 10   3 Admitted (0.3000000 0.7000000) *
#>     3) gpa>=3.415 192  82 Rejected (0.5729167 0.4270833)
#>       6) rank=2,3,4 160  58 Rejected (0.6375000 0.3625000)
#>        12) rank=3,4 89  27 Rejected (0.6966292 0.3033708) *
#>        13) rank=2 71  31 Rejected (0.5633803 0.4366197)
#>          26) gpa>=3.495 55  20 Rejected (0.6363636 0.3636364)
#>            52) gpa< 3.73 26   5 Rejected (0.8076923 0.1923077) *
#>            53) gpa>=3.73 29  14 Admitted (0.4827586 0.5172414)
#>             106) gre>=690 9   3 Rejected (0.6666667 0.3333333) *
#>             107) gre< 690 20   8 Admitted (0.4000000 0.6000000) *
#>          27) gpa< 3.495 16   5 Admitted (0.3125000 0.6875000) *
#>       7) rank=1 32   8 Admitted (0.2500000 0.7500000) *
```

```
round(tree_adm$variable.importance, 2)   # total impurity reduction (incl. surrogate splits)
```

```
#>   gpa  rank   gre
#> 14.65 12.51  7.49
```

```
rpart.plot(tree_adm, type = 2, extra = 106, under = TRUE, faclen = 0,
           box.palette = "BuGn", tweak = 1.05,
           main = "Admission decision tree (number in box = P(Admitted), % = share of applicants)")
```

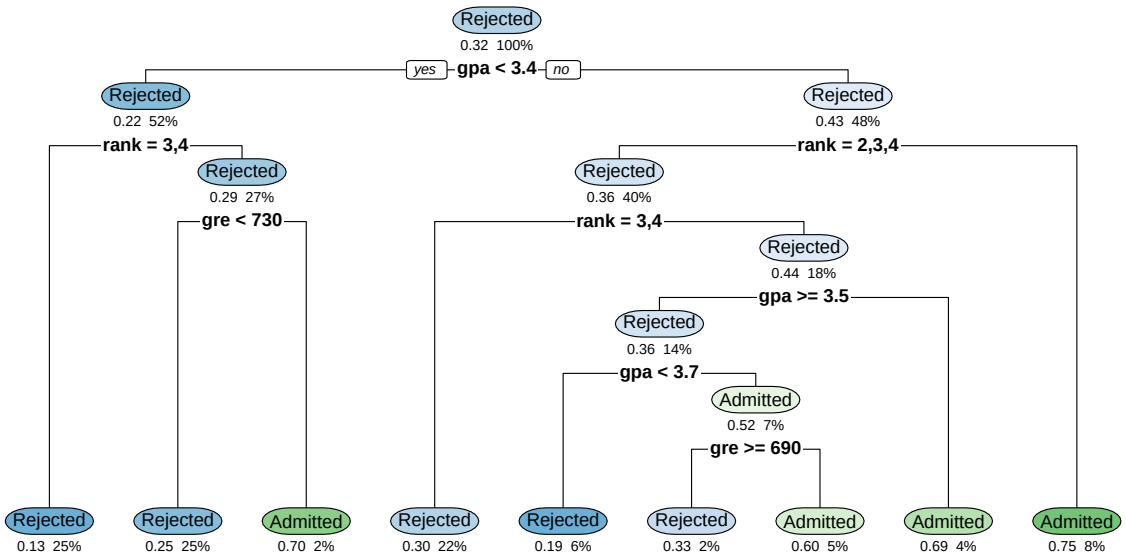


Figure 2: Classification tree for admission (rpart default settings, cp = 0.01). Each box shows the predicted class, P(Admitted) and the share of applicants in that node.

Structure of the tree. rpart grows the tree by repeatedly choosing the split that most reduces the Gini impurity. Each leaf predicts the majority class among its applicants, which is the same as using a 0.5 probability threshold. The tree has 8 splits and 9 leaves.

1. **Root split, GPA < 3.415.** This is the single most informative variable. It divides applicants into a low-GPA group (208 applicants, 21.6% admitted) and a high-GPA group (192 applicants, 42.7% admitted).
2. **Low-GPA branch.**
   - Applicants from rank 3–4 schools are rejected (13.1% admitted).
   - Applicants from rank 1–2 schools are admitted only if their GRE is at least 730 (70% admitted, only 10 applicants). Otherwise they are rejected (25.3%).
3. **High-GPA branch.**
   - Rank 1 applicants are **admitted** (75% admitted, 32 applicants).
   - Rank 3–4 applicants are **rejected** (30.3%).
   - Rank 2 applicants are split further on GPA and GRE: GPA $< 3.495 \rightarrow$ admitted (68.8%); $3.495 \leq$ GPA $< 3.73 \rightarrow$ rejected (19.2%); GPA ≥ 3.73 and GRE $< 690 \rightarrow$ admitted (60%); GPA ≥ 3.73 and GRE $\geq 690 \rightarrow$ rejected (33.3%).

Reading the tree. A high GPA plus a top-ranked institution is the clearest route to admission. GPA and rank do most of the work, and GRE matters mainly at the margin. The deepest rank-2 splits are based on very small nodes (9 to 29 applicants) and partly contradict common sense: a *higher* GRE leads to rejection, and a middling GPA does worse than a lower one. This is a typical sign that the tree is fitting noise, which part (e) confirms.

(e) Examining the tree with `printcp`

```
# (e) Complexity-parameter table
printcp(tree_adm)
```

```
#>
#> Classification tree:
#> rpart(formula = admit ~ gpa + rank + gre, data = adm, method = "class")
#>
#> Variables actually used in tree construction:
#> [1] gpa  gre  rank
#>
#> Root node error: 127/400 = 0.3175
#>
#> n= 400
#>
#>         CP nsplit rel error  xerror     xstd
#> 1 0.062992      0   1.00000 1.00000 0.073308
#> 2 0.023622      2   0.87402 0.95276 0.072337
#> 3 0.015748      4   0.82677 1.01575 0.073612
#> 4 0.010000      8   0.76378 1.02362 0.073760
```

```
cp_tab   <- as.data.frame(tree_adm$cptable)
best_row <- which.min(cp_tab$xerror)
cp_min   <- cp_tab$CP[best_row]                                    # minimum-xerror rule
se_limit <- cp_tab$xerror[best_row] + cp_tab$xstd[best_row]
cp_1se   <- cp_tab$CP[which(cp_tab$xerror <= se_limit)[1]]         # one-standard-error rule
root_err <- mean(adm$admit == "Admitted")                          # root node error = 127/400

cp_tab$train_error_pct <- 100 * root_err * cp_tab$`rel error`
cp_tab$cv_error_pct    <- 100 * root_err * cp_tab$xerror
round(cp_tab, 4)
```

```
#>       CP nsplit rel error xerror   xstd train_error_pct cv_error_pct
#> 1 0.0630      0    1.0000 1.0000 0.0733           31.75        31.75
#> 2 0.0236      2    0.8740 0.9528 0.0723           27.75        30.25
#> 3 0.0157      4    0.8268 1.0157 0.0736           26.25        32.25
#> 4 0.0100      8    0.7638 1.0236 0.0738           24.25        32.50
```

```
c(cp_min = cp_min, cp_1se = cp_1se)
```

```
#>     cp_min     cp_1se
#> 0.02362205 0.06299213
```

```
tree_pruned <- prune(tree_adm, cp = cp_min)
print(tree_pruned)
```

```
#> n= 400
#>
#> node), split, n, loss, yval, (yprob)
#>       * denotes terminal node
#>
```

```
#> 1) root 400 127 Rejected (0.6825000 0.3175000)
#>   2) gpa< 3.415 208  45 Rejected (0.7836538 0.2163462) *
#>   3) gpa>=3.415 192  82 Rejected (0.5729167 0.4270833)
#>     6) rank=2,3,4 160  58 Rejected (0.6375000 0.3625000) *
#>     7) rank=1 32   8 Admitted (0.2500000 0.7500000) *
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 6.5, 1))
plotcp(tree_adm)      # top axis = number of leaves; dotted line = min xerror + 1 SE
title("Cross-validated error vs cp", line = 5, cex.main = 0.95)
rpart.plot(tree_pruned, type = 2, extra = 106, under = TRUE, faclen = 0,
           box.palette = "BuGn", main = "Pruned tree (cp with minimum xerror)",
           cex.main = 0.95)
```

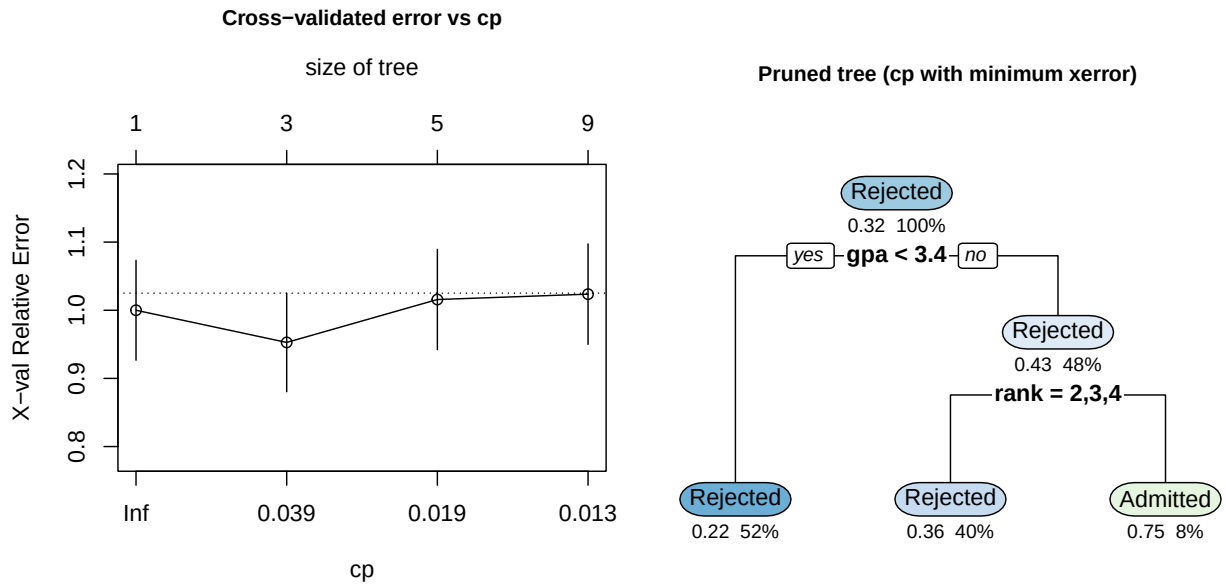


Figure 3: Left: cross-validated relative error (+/-1 SE) against cp. Right: tree pruned at the cp with the lowest cross-validated error.

```
par(mfrow = c(1, 1))
```

What the columns mean.

- **Root node error** = 127/400 = 0.3175. This is the error from predicting "Rejected" for everyone, and all other errors are expressed relative to it.
- **CP** is the complexity parameter. In cost-complexity pruning a split is kept only if it lowers the relative error by at least CP. Equivalently, the tree minimises Error$(T) + cp \times |T| \times$ root error, where $|T|$ is the number of splits.
- **nsplit** is the number of splits, so the number of leaves is nsplit + 1.
- **rel error** is the *training* error relative to the root.
- **xerror** is the 10-fold *cross-validated* error relative to the root, and **xstd** is its standard error.

What the table shows.

- **Training error keeps falling as the tree grows.** rel error goes from 1 to 0.764, so the full 8-split tree misclassifies 24.25% of the training applicants.
- **Cross-validated error is lowest at 2 splits** (xerror = 0.953, a CV error rate of 30.25%). It then *rises* to 1.024 for the full tree (32.50%), which is worse than not splitting at all. **The extra splits from part (d) overfit.**

- **The predictive power is weak.** Every xerror lies within about one standard error (≈ 0.073) of 1. Even the best tree beats the "reject everyone" rule (31.75% error) by only 1.5 percentage points.

How to choose the most suitable cp.

- **Minimum-xerror rule.** Pick the cp with the smallest xerror, here **cp = 0.0236** (2 splits), and run `prune(tree, cp = ...)`.
- **One-standard-error rule.** Pick the *simplest* tree whose xerror is below min(xerror) + its xstd, here 1.025. That gives cp = 0.0630, which is the root with no split. It would predict that nobody is admitted, so it is useless for this purpose.

We therefore use the minimum-xerror cp. The pruned tree (Figure 3, right) is simple: **GPA $<$ 3.415 $\rightarrow$ rejected; GPA $\geq$ 3.415 and rank 1 $\rightarrow$ admitted; otherwise rejected.**

Because xerror comes from random CV folds, it changes with the seed. We fix `set.seed(2026)` for reproducibility. In practice one could repeat the cross-validation several times, or set `xval` higher, before settling on cp.

(f) Applicant with GPA = 3.8, GRE = 690, rank 4

```
# (f) Applicant with GPA = 3.8, GRE = 690 from a rank-4 institution
new_applicant <- data.frame(gpa = 3.8, gre = 690, rank = factor(4, levels = 1:4))
q3f <- data.frame(
  tree          = c("Full tree from (d)", "Pruned tree from (e)"),
  P_admitted    = c(predict(tree_adm,    new_applicant, type = "prob")[, "Admitted"],
                    predict(tree_pruned, new_applicant, type = "prob")[, "Admitted"]),
  decision      = c(as.character(predict(tree_adm,    new_applicant, type = "class")),
                    as.character(predict(tree_pruned, new_applicant, type = "class")))
)
q3f
```

```
#>                   tree P_admitted decision
#> 1   Full tree from (d)  0.3033708 Rejected
#> 2 Pruned tree from (e)  0.3625000 Rejected
```

Following the tree from part (d):

1. GPA = 3.8 $\geq$ 3.415, so we go to the high-GPA branch.
2. Rank 4 is not rank 1, so we move into the rank 2–4 node.
3. Rank 4 belongs to the rank 3–4 group, which is a leaf with P(Admitted) = 0.303.

The tree predicts that this applicant will *not* be admitted. In the training data only about 30% of high-GPA applicants from rank 3–4 schools were accepted. GRE is never used on this path, so the good GRE score of 690 does not change the outcome. The pruned tree from part (e) gives the same answer (P(Admitted) = 0.362).

Question 4: ROC curves

How an ROC curve is created.

1. A classifier gives each observation a score, usually the predicted probability $\hat{p}$ of the positive class. An observation is classified as positive if $\hat{p} \geq t$ for some threshold t.

2. For each threshold we compute the **true positive rate** $TPR = TP/(TP + FN)$ (sensitivity) and the **false positive rate** $FPR = FP/(FP + TN)$ (1 - specificity).
3. We sweep t from 1 down to 0. At $t = 1$ nothing is classified positive, giving the point $(0, 0)$. At $t = 0$ everything is positive, giving $(1, 1)$. Plotting all the (FPR, TPR) pairs gives the ROC curve.
4. The 45° diagonal corresponds to random guessing. The **area under the curve (AUC)** equals the probability that a randomly chosen positive case receives a higher score than a randomly chosen negative case: 0.5 is random and 1 is perfect.

How we use ROC curves in data analysis.

- **Comparing models without fixing a threshold.** AUC summarises how well a model *ranks* cases, and it does not depend on the 0.5 cut-off or on the class mix. It is therefore more informative than accuracy when classes are imbalanced (Question 2). If one curve lies above another everywhere, that model is better at every threshold.
- **Choosing the operating threshold.** The curve shows the full trade-off between catching positives and raising false alarms. We pick the point that fits the costs of the problem. A credit-risk team might require TPR $\geq 90\%$ and accept the FPR that comes with it. A marketing campaign with a limited budget might cap the FPR. Without explicit costs, Youden's index (max TPR - FPR) is a common default.
- **Diagnosing the model.** A curve close to the diagonal means the features carry little signal; the stock models in Question 6 have AUC $\approx$ 0.68–0.73. An AUC that is suspiciously close to 1 is a warning sign of data leakage. ROC curves should always be computed on test or cross-validation data, never on the training data.

Interpreting the graph.

- **Both models are far better than chance.** Logistic regression has AUC = 0.965 and the decision tree has AUC = 0.902.
- **The logistic curve is the better one.** It is a fine staircase because logistic regression produces a continuous probability, so almost every threshold gives a different point. It rises almost vertically: about 72% of positives are caught with essentially no false positives, about 83% at FPR $\approx$ 0.03, about 95% at FPR $\approx$ 0.15, and 100% at FPR $\approx$ 0.45.
- **The tree curve has very few points.** It is made of straight segments with essentially one interior corner (FPR $\approx$ 0.03, TPR $\approx$ 0.83). A tree can only output as many distinct probabilities as it has leaves, so there are only a few usable thresholds. Between them the ROC is a straight-line interpolation, which suggests this tree is very small.
- **Comparison.** At a very low false-positive rate ($\approx$ 0.03) the two models perform about the same. At every higher FPR logistic regression dominates; at FPR = 0.15, for example, it detects about 95% of positives against about 87% for the tree.
- **Conclusion.** Logistic regression is the better classifier here. It has the higher AUC, is better at nearly every threshold, and gives much more freedom to choose the threshold. The tree would only be competitive if we had to operate at a very low false-positive rate.

Question 5

(a) Linear probability model (LPM) vs. logistic model.

1. **The range of predicted probabilities differs.**
   - The LPM models $P(Y = 1 \mid X) = X\beta$ directly as a straight line, so fitted "probabilities" can fall below 0 or above 1.

- The logistic model passes $X\beta$ through the sigmoid, $P(Y = 1 \mid X) = e^{X\beta}/(1 + e^{X\beta})$, so predictions always lie in $(0, 1)$.

2. **Marginal effects and interpretation differ.**
   - In the LPM the effect of x_j is constant: a one-unit increase changes the probability by β_j whatever the starting level.
   - In the logistic model the effect on the probability is $\beta_j\, p(1-p)$, which depends on x. It is largest around $p = 0.5$ and small in the tails (the S-shape). The coefficient β_j is the change in the *log-odds*, and e^{β_j} is an odds ratio.
3. **Estimation also differs.** The LPM is estimated by OLS and its errors are heteroskedastic by construction, since $\text{Var} = p(1-p)$. The logistic model is estimated by maximum likelihood.

(b) Two assumptions of LDA.

1. **Normality.** Within each class k, the predictors follow a multivariate normal distribution, $X \mid Y = k \sim N(\mu_k, \Sigma)$, with a class-specific mean vector μ_k.
2. **Common covariance.** All classes share the same covariance matrix Σ.

Under these two assumptions the log posterior odds are linear in x, so LDA produces linear decision boundaries. The class priors π_k are estimated from the class proportions.

(c) Main difference between LDA and QDA.

- **QDA drops the common-covariance assumption.** Each class has its own covariance matrix Σ_k, so the discriminant function is *quadratic* in x and the decision boundaries can be curved.
- **The cost is many more parameters to estimate:** $Kp(p+1)/2$ covariance parameters instead of $p(p+1)/2$. QDA is therefore more flexible, with lower bias but higher variance. It suits large samples or classes whose covariance structures clearly differ.
- **LDA is often better when n is small** or the covariances are similar.

In Question 6, QDA had the lowest cross-validated accuracy of all the models, which is consistent with its higher variance and with heavy-tailed, non-normal financial ratios.

Question 6: Predicting the 2019 stock-price direction

(a) Finding the most accurate model

Data preparation.

- The data set has 4,392 companies, 13 financial ratios and the target y (1 = price higher in 2019). The company name is an identifier and is not used as a predictor.
- 252 companies were missing more than half of the 13 ratios and were removed, leaving **4,140 companies**. Of these, 68.2% rose in 2019.
- The data were split into a stratified 70% training set (2,897) and a 30% test set (1,243).
- The ratios contain extreme outliers (for example, return on equity reaches 1.1×10^7) and some missing values (return on assets is missing for 20.6% of firms). Using the **training data only**, we filled missing values with the median and winsorised each ratio at its 1st and 99th percentiles. The same transformation was then applied to the test data, so no information leaks from the test set.

Models compared. Because y is a 0/1 variable, we used classification methods. We compared the **seven classification methods taught in the course**, all using the 13 ratios. The linear

probability model was added only as a reference benchmark, because its fitted values can fall outside $[0, 1]$.

Lecture	Model	Settings
6	Logistic regression	Maximum likelihood
6	LDA	Common covariance matrix
6	QDA	Class-specific covariance matrices
8	Classification tree	Grown with cp = 0.001, then pruned at the cp with the lowest xerror
9	Bagging	500 bootstrap trees, all 13 ratios tried at each split ($m_{try} = 13$)
9	Random forest	500 trees, $m_{try} = 3$ randomly chosen ratios per split
9	KNN	Standardised ratios; $k = 33$ chosen by 10-fold CV over $k = 1, 3, \ldots, 101$
6	LPM (benchmark only)	OLS on the 0/1 target

Every model classifies a stock as "Up" when $\hat{p} > 0.5$. Models were **ranked by 10-fold cross-validated accuracy on the training set**, using the same folds for every model, and then **checked once on the untouched test set** (Lecture 7).

Table 3: Model comparison, sorted by CV accuracy (accuracy, sensitivity and specificity in %; positive class = price up; threshold 0.5)

Model	Lecture	CV acc.	CV s.d.	Test acc.	Sensitivity	Specificity	Test AUC
Random forest	L9	70.5	1.6	69.8	86.3	34.2	0.732
LPM (benchmark)	L6	70.1	1.6	70.2	95.6	15.4	0.700
LDA	L6	69.9	1.8	70.5	94.9	18.0	0.700
Bagging	L9	69.9	1.9	68.7	83.3	37.5	0.730
Logistic regression	L6	69.9	1.9	70.2	95.0	17.0	0.700
KNN	L9	69.6	2.2	70.1	92.2	22.5	0.713
Classification tree	L8	69.4	2.9	69.3	86.2	33.2	0.677
QDA	L6	68.5	2.1	68.9	88.3	27.1	0.693
Baseline: always predict Up	-	68.2	2.0	68.2	100.0	0.0	0.500

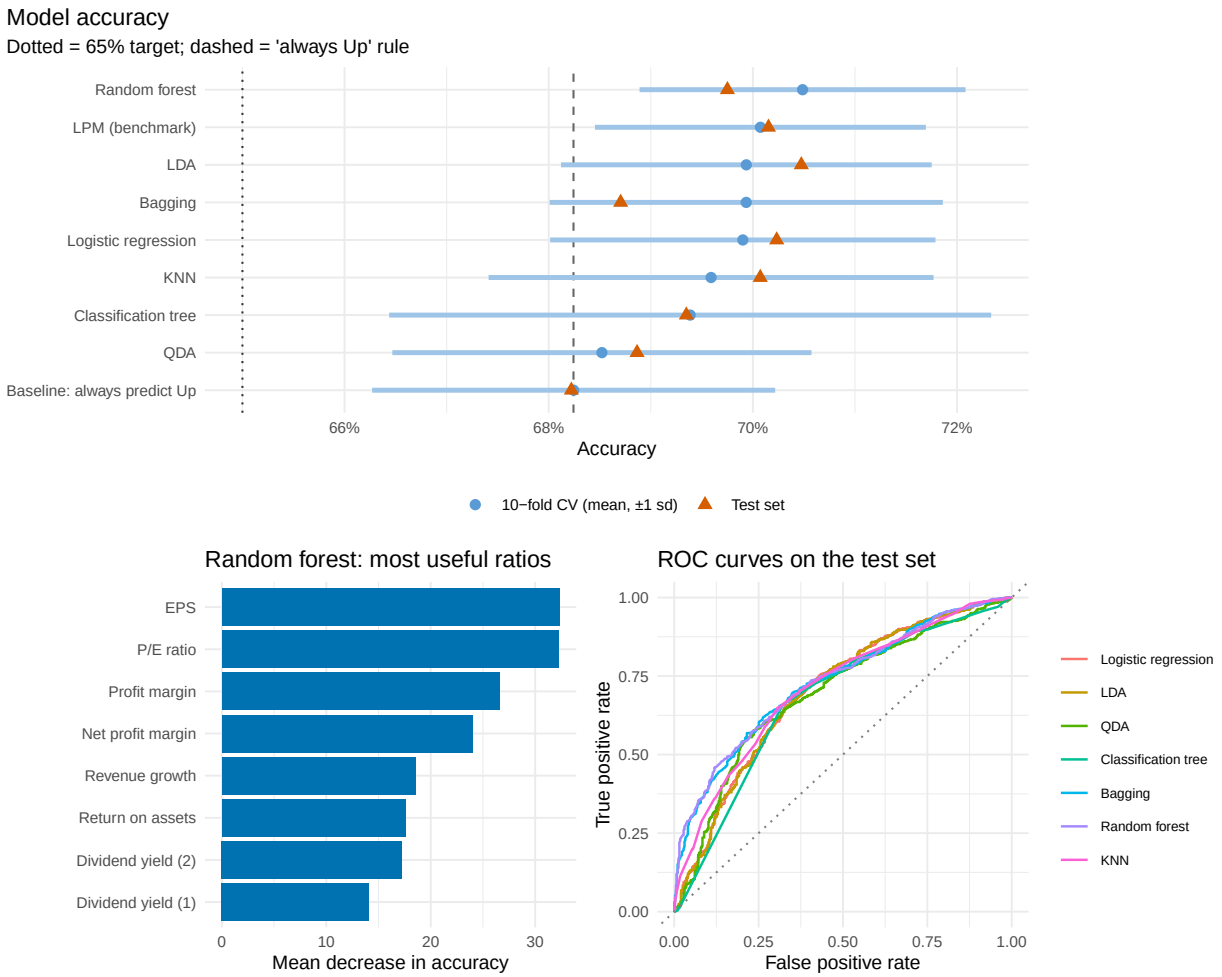


Figure 4: Top: accuracy of each model (10-fold CV mean +/- 1 s.d. and test set); dotted line = 65% target, dashed line = naive rule. Bottom: random-forest variable importance and ROC curves of the seven course models on the test set.

All models meet the 65% accuracy requirement.

- Every course model is above 65% in cross-validation (lowest: QDA, 68.5%) and on the test set (lowest: Bagging, 68.7%).
- The tougher benchmark is the naive rule "always predict Up", which already scores 68.2% in cross-validation and 68.2% on the test set.

Result: the random forest is the best model.

- **Accuracy.** It has the highest cross-validated accuracy, **70.5%**. Cross-validation is the basis for choosing the model; the test set is only used to confirm the choice.
- **Test set.** On the test set it reaches **69.8%**, with AUC 0.732. The LDA scores slightly higher on the test set (70.5%), but differences of this size are within noise. Refitting the random forest with 10 different seeds gives test accuracies between 69.3% and 70.7% (mean 70.1%).
- **Tree → bagging → random forest.** CV accuracy rises from 69.4% for a single pruned tree, to 69.9% for bagging, to 70.5% for the random forest.
  - Averaging many bootstrap trees reduces variance (Question 1).
  - Letting each split consider only 3 random ratios makes the trees less correlated, so the average reduces variance further.

  - Across the 10 seeds the random forest beats bagging on the test set on average (70.1% against 69.1%).
- **Most useful predictors.** EPS and the P/E ratio top the importance ranking, followed by the two profit margins. P/E is recorded as 0 for every loss-making company, so both variables mainly capture *whether a company is profitable*.
- **Confusion matrix on the test set.** Rows are predictions and columns are actual outcomes:

Table 4: Test-set confusion matrix of the random forest

	Down	Up
Down	135	116
Up	260	732

Two caveats apply:

- **The differences between the top models are small.** Logistic regression, LDA, bagging and KNN are all within about one cross-validation standard deviation of the random forest, so they are statistically similar.
- **All models are only slightly better than the naive rule.** On the test set the random forest beats "always predict Up" by 1.5 percentage points. This gain is not statistically significant at the 5% level (one-sided binomial test, $p = 0.130$). Part (c) discusses what this means.

(b) One-page report for the supervisor

To: Supervisor | **From:** Group ____ | **Date:** October 2026
Subject: Can 2018 financial ratios predict which US stocks rise in 2019?

Data. We analysed 13 financial ratios from 2018 annual reports for 4,392 US-listed companies. 252 companies with most ratios missing were dropped, leaving 4,140. **68.2% of these stocks rose in 2019**, so a naive "everything goes up" rule is already right 68.2% of the time, and any useful model must beat that benchmark. The ratios are extremely skewed (ROE reaches 1.1×10^7), so Table 5 compares medians and uses the Wilcoxon rank-sum test.

Table 5: Median of each 2018 ratio, for stocks that fell or rose in 2019

Ratio	Fell	Rose	Wilcoxon p	Ratio	Fell	Rose	Wilcoxon p
Revenue growth	0.0561	0.0787	0.005	Debt ratio	0.172	0.221	< 0.001
EPS	-0.08	1.2	< 0.001	P/E ratio	0	12.5	< 0.001
Profit margin	-0.002	0.067	< 0.001	Dividend yield (2)	0	0.008	< 0.001
Net profit margin	0	0.0696	< 0.001	Book value/share growth	-0.0126	0.0376	< 0.001
Dividend yield (1)	0	0.00875	< 0.001	Debt growth	0	0	< 0.001
Return on assets	-0.0108	0.0732	< 0.001	R&D expense growth	0	0	0.142
Return on equity	0.0057	0.0907	< 0.001				

Descriptive findings. Stocks that rose were typically **profitable dividend payers**. Median EPS was 1.20 for rising stocks against -0.08 for falling ones, and median profit margin was 6.7% against -0.2%. 77% of profitable companies rose, compared with 51% of loss-making ones (Figure 5, left). 12 of the 13 ratios differ significantly at the 1% level; R&D-expense growth does not. Several ratios are near-duplicates, such as the two dividend yields and the two profit margins (rank correlation above 0.9).

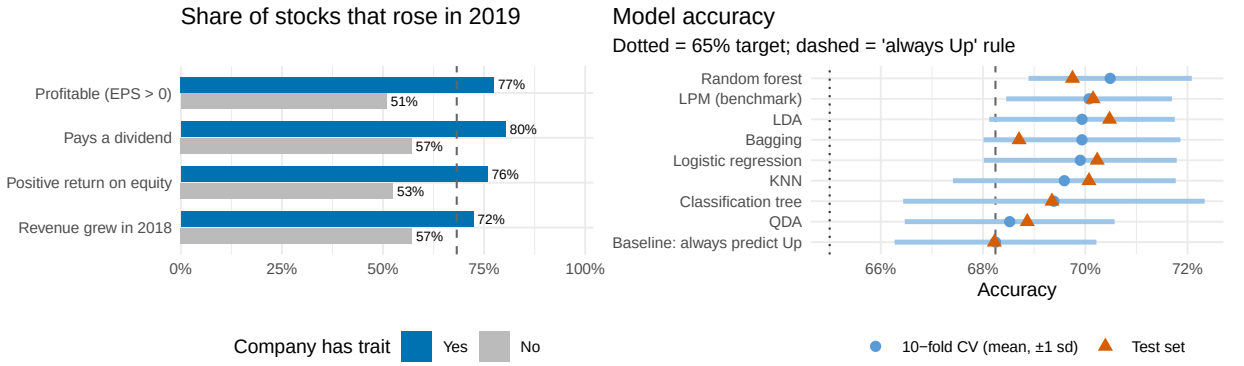


Figure 5: Left: share of stocks that rose in 2019, for companies with and without each trait (dashed line = all companies). Right: model accuracy against the 65% target (dotted) and the naive benchmark (dashed).

Model results. We compared the seven classification methods taught in the course: logistic regression, LDA, QDA, a classification tree, bagging, a random forest and KNN. Each was trained with 10-fold cross-validation on 70% of the data and confirmed on the remaining 30%. **All of them exceed the 65% accuracy target. The random forest performed best in cross-validation** (70.5%), with 69.8% test accuracy and an AUC of 0.73. Its most useful inputs were EPS, the P/E ratio and the profit margins.

Bottom line. On the test set the best model beats the naive benchmark by only 1.5 percentage points, and it correctly flags just 34% of the stocks that fell. 2018 accounting ratios on their own carry only weak information about the direction of 2019 prices. **We do not recommend using the model for investment decisions as it stands.** The next steps are to add market and sector data, to predict returns *relative to the market*, and to tune the models further (see 6(c)).

(c) Is the best model good enough?

It meets the 65% accuracy target, but it is not good enough to be useful. Several facts point the same way:

- **Little improvement over the naive rule.** The random forest's test accuracy (69.8%) is only 1.5 points above always predicting "Up" (68.2%), and the difference is not statistically significant (p = 0.130). Its cross-validated accuracy is only about 2.2 points above the naive rule's.
- **Poor detection of falling stocks.** Specificity is 34.2%, so about 66% of the stocks that fell are still predicted to rise.
- **Modest ranking ability.** An AUC of 0.73 is modest by any standard.

This is not surprising. A stock's price direction over a full year is driven largely by market-wide movements (2019 was a strong year for US equities, which is why about two-thirds of stocks rose) and by news that arrives *after* the 2018 accounts are published. Publicly available accounting ratios are also largely priced in already.

Possible ways to increase accuracy.

1. **Better and more features.** These are likely to help most.
   - Add market-based information: past price momentum and volatility, beta, market capitalisation, trading volume.
   - Add sector or industry dummies and *sector-relative* ratios.
   - Add more of the 200+ indicators in the original data source (valuation multiples, cash-flow measures, growth rates).
   - Add macroeconomic variables and analyst-revision or news-sentiment data.
2. **A better-defined target and more data.**
   - Predict whether a stock *outperforms the market or its sector*, which removes the common market drift that no firm-level ratio can explain.
   - Pool several years (for example 2014–2018) and validate on later years, so the model is trained and tested over different market conditions.
3. **Better tuning of the course models.**
   - Tune the random forest's m_{try}, node size and number of trees with out-of-bag error or cross-validation. A quick out-of-bag check over $m_{try} = 1, \ldots, 13$ moved accuracy by only about one point, so large gains are unlikely from this alone.
   - Select features for KNN and LDA, since irrelevant or duplicated ratios hurt distance-based and linear methods.
   - Average the predicted probabilities of several models, for example logistic regression, LDA and the random forest.
   - Methods beyond this course, such as boosting, could be tried later.
4. **Better data treatment.**
   - Missing-value indicators or multiple imputation, since missingness itself may be informative.
   - Log or rank transformations of the skewed ratios.
   - Removing redundant ratios, such as the duplicate dividend yields and profit margins.
5. **A threshold and metric that match the business goal (Lecture 7).**
   - If avoiding losers matters, choose the threshold from the ROC curve instead of 0.5, to raise specificity.
   - Judge the model by AUC, balanced accuracy or, ultimately, the portfolio return it produces, rather than raw accuracy, which mostly rewards predicting the majority class.

Appendix: Descriptive graphics for Question 6

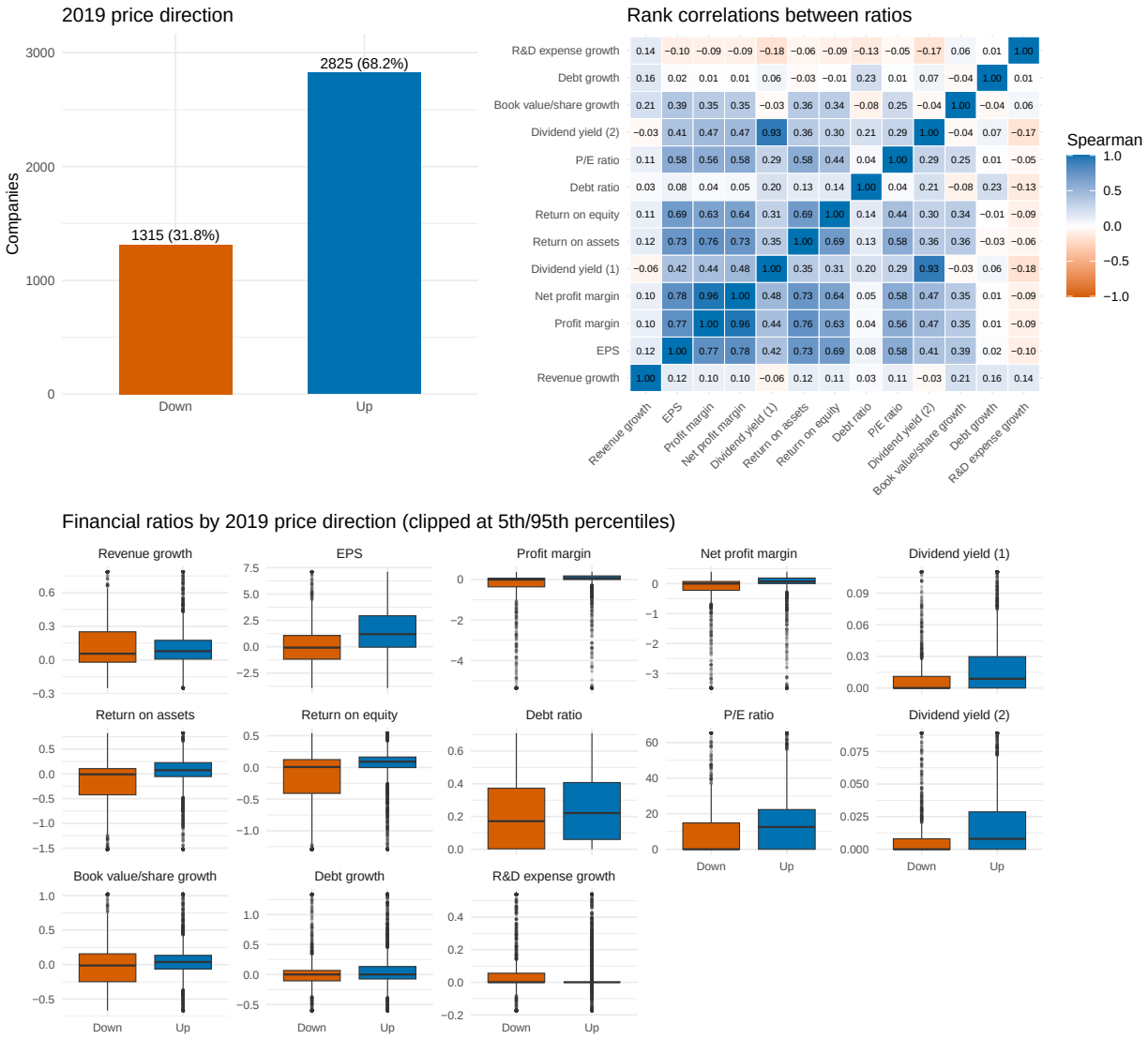


Figure 6: Class balance, rank correlations between the ratios, and the distribution of every ratio by 2019 price direction.